

ICSOTpentest.py v2.1

ICS/OT/IoT Penetration Testing & Threat Modeling Framework Advanced User Guide

Version 2.1 | April 2026
Copyright © April 2026 Isiah Jones. All Rights Reserved.

<https://isiahjonesstem.github.io/security-methodology/>
Contact: isiahjones@mail.com

11,160+ lines | 56 protocols | 121 native methods | 242+ functions | 66 benchmark checks | 39 network vendors | 8 compliance frameworks

Table of Contents

1. Introduction & Architecture
2. Installation & Requirements
3. Quick Start Guide
4. Phase 0: Planning & Scope
5. Phase 1: Reconnaissance
6. Phase 2: Enumeration
7. Phase 3: Vulnerability Analysis
8. Phase 4: Exploitation
9. Phase 5: Post-Exploitation
10. Phase 6: Threat Modeling
11. Phase 7: Mitigation Mapping
12. Phase 8: Reporting

13. Native Protocol Scanner Reference
14. STIG/CIS Benchmark Scanner Reference
15. Covert Communications & Exfiltration
16. Network Infrastructure Database
17. External Tool Integrations
18. Data Sources & Customization
19. Command-Line Interface Reference
20. AI-Assisted Analysis
21. Troubleshooting
22. Appendix: Complete Protocol & Port Reference

1. Introduction & Architecture

ICSOTpentest.py is a self-contained, professional-grade penetration testing and threat modeling framework purpose-built for Industrial Control Systems (ICS), Operational Technology (OT), and Internet of Things (IoT) environments. Version 2.1 represents a significant expansion with 11,160+ lines of pure Python code, 242+ functions, and 66 STIG/CIS benchmark compliance checks.

Key Design Principles

Self-contained: All reference data (228 credentials, 228 OSINT dorks, 62 pentest tasks, 47 attack test cases) embedded natively. Single-file deployment — no external JSON or config files needed.

Native-first: 121 pure Python scanner methods (60 enumeration + 56 exploitation + 5 utility) implementing actual ICS protocol handshakes using socket + struct. Native features ALWAYS execute first; external tools like Nmap and Metasploit augment if available.

Standards-aligned: MITRE ATT&CK for ICS, MITRE EMB3D, ISA/IEC 62443, NIST SP 800-82, NERC CIP, D3FEND, ISA 84.00.09, EU CRA, NIS2, AWWA. Plus DISA STIGs and CIS Benchmarks.

Report-ready: Native DOCX, PDF, HTML, Markdown, CSV, XLSX, SCAP/XCCDF generation. No pandoc, wkhtmltopdf, or LaTeX needed.

Resilient: All operations wrapped in error handling. Session auto-save after every action. Cancel any operation with Ctrl+C. Real-time activity logging to icsot_activity.log.

Architecture Overview

Component	Description	Lines
InteractiveConsole	Metasploit-style CLI with 9 kill chain phases, wizard, help, cancel support	~1,400
NativeScanner	121 protocol enum/exploit methods + ARP/SSL/DNS/OWASP native checks	~3,400
BenchmarkScanner	66 STIG/CIS/SCAP configuration compliance checks across 12 categories	~800
ProgressBar	Native Python progress bars for all long-running operations	~50
ActivityLogger	Real-time activity logging with timestamps and phase tracking	~50
PhaseExecutors	Execution engines for each assessment phase with automatic data flow	~1,700

ReportGenerator	Native DOCX/PDF/HTML/CSV/MD/XLSX/XCCDF report generation	~1,000
CovertCommsModule	8 tunnel types, steganography, 11 exfiltration destinations	~550
CVSSCalculator	CVSS v3.1 base score calculation with ICS-specific patterns	~100
Data/Config/Models	Protocol defs, MITIGATION_MAP (37 TTPs), NETWORK_INFRA (39 vendors)	~2,700

Native-First Priority Model

When both native Python methods and external tools are available for the same task, ICSOTpentest.py ALWAYS runs native methods first. External tools augment the native results — they never replace them. This ensures the tool works fully even on air-gapped ICS networks with no internet access and no additional software installed.

Task	Priority 1 (Native)	Priority 2 (External)	Priority 3
Port scanning	Native Python tcp_port_scan()	Nmap (if installed)	Results merged
SSH testing	Native socket banner grab	paramiko (if installed)	sshpas fallback
GOOSE capture	Native raw socket (AF_PACKET)	tshark (if installed)	—
CVE lookup	Native urllib NVD query	NVD API key (faster)	—
Shodan lookup	InternetDB API (free, no key)	Full Shodan API (with key)	—
ARP/Layer 2 scan	Native arp -a parsing	arp-scan (if installed)	—
SSL/TLS enum	Native ssl module	Nmap ssl-enum-ciphers	—
Web security checks	Native urllib probes	nikto/ZAP (if installed)	—
Report generation	Native Python stdlib	— (no external needed)	—

2. Installation & Requirements

Minimum Requirements

- Python 3.8 or higher
- pip install cryptography (required for license verification)
- Any OS: Windows 10/11, Linux (Ubuntu/Kali/RHEL), macOS
- No other pip packages required for core functionality

Step-by-Step Installation

Step 1: Verify Python version

```
python3 --version # Must be 3.8+ python --version # Windows PowerShell
```

Step 2: Install the cryptography library

```
pip install cryptography pip3 install cryptography # If pip points to Python 2
```

This is the only pip package required. It is used for license file verification. All other functionality uses Python standard library.

Step 3: Place the script and license file

Copy ICSOTpentestv2.py and your .lic license file into the same directory. The script is a single file (~600 KB) containing all embedded data.

Step 4: Launch the tool

```
python3 ICSOTpentestv2.py # Linux/macOS python ICSOTpentestv2.py # Windows PowerShell
python3 ICSOTpentestv2.py --no-color # Terminals without ANSI support
```

License Activation

You will receive a .lic license file with your purchase. The tool verifies it automatically on every run. If you need to store the license file in a different location, set the ICSOT_LICENSE_FILE environment variable:

```
# Windows PowerShell $env:ICSOT_LICENSE_FILE = "C:\path\to\your\license.lic" # Linux/macOS
export ICSOT_LICENSE_FILE=/path/to/your/license.lic
```

■ **Do not share your .lic file. Each license is cryptographically signed and tied to your purchase. Tampering with the license file will invalidate it.**

Optional Enhancements

Tool	What It Adds	Install Command
Nmap	Deep service detection, NSE scripts	apt install nmap
Metasploit	Exploit module execution	apt install metasploit-framework
Graphviz	Render kill chain diagrams to PNG/SVG	apt install graphviz
tshark	PCAP analysis, GOOSE/SV capture	apt install tshark
theHarvester	OSINT email/subdomain harvesting	pip install theHarvester
paramiko	Pure Python SSH for credential testing	pip install paramiko
openpyxl	XLSX credential import + XLSX report output	pip install openpyxl

All integrations are optional. The tool functions fully without them.

3. Quick Start Guide

Step 1: Launch the Tool `python3 ICSOTpentestv2.py`

You will see the ASCII banner, legal disclaimer, embedded data counts (credentials, dorks, tasks), and the main menu with phases 0-8.

Step 2: Run the Setup Wizard (Recommended First Time) — Type W at the main menu. The wizard walks you through: (1) Adding target IP addresses, CIDR ranges, or IP ranges, (2) Adding URL targets for web-based HMI/config interfaces, (3) Selecting which of the 56 ICS protocols to test (or select all), (4) Setting engagement details (project name, customer name, dates, engagement type), (5) Configuring optional API keys, (6) Setting scan safety parameters (aggression level 1-5, rate limits, timeouts).

■ *For production ICS environments, use aggression level T1 or T2. T3+ may cause device resets.*

Step 3: Run the Assessment — Type A to run all 9 phases automatically (good for initial scans), or type any number 0-8 to enter a specific phase. Each phase is independent — you can skip ahead or go back at any time but some later phases have data dependencies from previous phases.

Step 4: Generate Reports — Phase 8 > Option 5 generates all report formats simultaneously (MD, HTML, DOCX, PDF). All generated natively — no external tools needed. Option 6 exports the MITRE ATT&CK/EMB3D attack mapping matrix as XLSX.

Navigation Shortcuts (Available in All Menus)

Key	Action	Where It Works
B	Go back to parent menu / main menu	Every menu and sub-menu
H	Show help text for current phase	Phase menus
Q	Quit the tool (with confirmation)	Main menu and phase menus
Ctrl+C	Cancel current operation gracefully, return to menu	Everywhere (will not crash)
V	View real-time activity log (last 30 entries)	Every menu
L	Load/restore a previous session	Main menu
targets	Show configured targets + discovered assets	Any phase menu
target add IP	Add a new target IP on-the-fly	Any phase menu
target edit IP	Edit target metadata (vendor, MAC, ports)	Any phase menu
A	Run all phases automatically	Main menu
W	Launch setup wizard	Main menu
S	Show engagement status summary	Main menu
T	Show available tools and integration status	Main menu
D	Show credential database stats	Main menu
P	List all 56 supported protocols	Main menu
F	Show current findings list	Main menu

set/show Commands (MSF-Style)

The interactive console supports Metasploit-style set and show commands for quick configuration:

```
set target 10.10.1.0/24      # Set target range set shodan_key YOUR_KEY      # Set Shodan
API key set nvd_key YOUR_KEY      # Set NVD API key set ai_key YOUR_KEY      # Set
```

```

Claude/OpenAI API key set aggression 2          # Set scan aggression show targets
# Display configured targets show findings        # Display current findings show
protocols          # List supported protocols show tools          # Show
external tool status show creds          # Show credential database

```

Phase Prerequisites and Skipping

Phase	Standalone?	What It Needs
Phase 0 (Planning)	Yes	Nothing — always available
Phase 1 (Recon)	Yes	Target IPs from Phase 0 (or enter when prompted)
Phase 2 (Enumeration)	Needs assets	Run Phase 1 first, OR manually add IPs in Phase 0
Phase 3 (Vuln Analysis)	Partially	Options 5 (Nessus Import) and 7 (Benchmark) work standalone
Phase 4 (Exploitation)	Needs assets	Option 3 (Protocol Exploits) lets you enter IPs directly
Phase 5 (Post-Exploit)	Needs exploits	Requires earlier phases for meaningful results
Phase 6 (Threat Model)	Needs findings	Run Phases 1-4 first to generate findings
Phase 7 (Mitigation)	Partially	Framework reference (opts 1-7) works; crosswalk needs findings
Phase 8 (Reporting)	Needs findings	Will warn and ask before generating empty reports

TIP: If you try to run a phase without the required data, the tool shows a clear warning message explaining what is missing and which earlier phase to run.

Command-Line Mode (Scripted/Automated)

```

python3 ICSOTpentestv2.py --target 10.10.1.0/24 --protocols modbus_tcp,s7comm,bacnet --
auto python3 ICSOTpentestv2.py --list-protocols          # Show all 56 protocols
python3 ICSOTpentestv2.py --list-creds                  # Show 228 default credentials
python3 ICSOTpentestv2.py --data-file extra.json        # Supplement embedded data
python3 ICSOTpentestv2.py --os-detection --aggression 2 # With OS detection python3
ICSOTpentestv2.py --full-port-scan                      # Scan all 65535 TCP ports python3
ICSOTpentestv2.py --resume session.json                # Resume previous session python3
ICSOTpentestv2.py --debug --log-file pentest.log        # Debug mode with custom log

```

4. Phase 0: Planning & Scope

Phase 0 configures the entire engagement before any scanning begins. This is the only phase that has no prerequisites.

Op t	Function	Details
1	Add IP Target	Single IP (10.10.1.25), CIDR (10.10.1.0/24), or range (10.10.1.1-50). Add multiple by running repeatedly.
2	Add URL/URI Target	http(s):// URLs for HMI, config portals, engineering workstations. Used for web-based testing in Phase 4 Option 8.
3	Add Hardware Target	Serial port paths for JTAG, UART, SWD, I2C, SPI, USB (e.g., /dev/ttyUSB0, COM3). Sub-menu lets you choose interface type.
4	Select Protocols	Choose specific protocols from all 56 supported, or press Enter to test all. Controls which native scanners and NSE scripts run.
5	Set Engagement Details	Project name, customer name, consultant name, dates, engagement type. Used in report headers, metadata, and output directory naming.
6	Configure API Keys	Shodan, Censys, NVD, AI API key (Claude/OpenAI), CALDERA server URL + key. All optional.
7	Load Custom Payload	Load Python (.py), Scapy, MSF RC (.rc), or NSE (.nse) scripts. Must have execute(target, port, config) or craft_packet().
8	Load Additional Creds	Import XLSX/CSV/JSON credential files to supplement the 228 embedded credentials. Requires openpyxl for XLSX.
9	Set Scan Options	Aggression level (1-5), max scan rate, timeout per host, UDP scanning, OS detection, full port scan toggle.
10	Set Output Options	Output directory path, report formats, screenshot capture, benign flag file settings.
W	Setup Wizard	Guided walkthrough of the most important settings. Recommended for first use.
R	Review Config	Displays all current settings: targets, protocols, API keys (masked), scan parameters, output settings.

Hardware Target Sub-Menu

When selecting Option 3, a sub-menu appears allowing you to choose the hardware interface type: (1) JTAG — Joint Test Action Group debug interface, (2) UART/Serial — Universal Async Receiver-Transmitter, (3) SWD — Serial Wire Debug for ARM processors, (4) I2C — Inter-Integrated Circuit bus, (5) SPI — Serial Peripheral Interface bus, (6) USB — Universal Serial Bus, (7) Custom — any other hardware interface. After selecting the type, enter the device path (e.g., /dev/ttyUSB0 on Linux, COM3 on Windows).

■ **ICS Safety: Use aggression level T1-T2 for production environments. T3+ may cause device resets or communication disruptions. Always coordinate with plant operations staff before scanning.**

TIP: You can return to Phase 0 at any time to add more targets, change protocols, or adjust settings. Changes apply immediately to subsequent phases.

5. Phase 1: Reconnaissance

Phase 1 discovers network assets, open ports, and attack surface using 10 different methods. Every method that can run natively does so by default.

Op t	Function	Native?	External Fallback
1	Active Network Scan	YES (native TCP scanner + progress bars)	Nmap augments if installed
2	Passive Network Discovery	YES (native ARP scan)	tshark for LLDP/CDP
3	Shodan Lookup	YES (InternetDB API — free)	Full Shodan API with key
4	OSINT Dork Search	YES (target-relevant search)	DuckDuckGo (no key needed)
5	DNS/Whois Recon	YES (socket + subdomain brute)	Nmap whois scripts
6	SNMP Discovery	YES (native ASN.1/BER scanner)	snmpwalk augments (optional)
7	Banner Grabbing	YES (raw socket)	None needed
8	Import Scan Results	YES (XML parsing)	Nmap XML, Nessus, PCAP
9	OSINT Harvesters	No	theHarvester, subfinder, httpx
10	Native ICS Protocol Scan	YES (most powerful option)	None needed — 60+ ports
R	Run All Recon	YES	Runs 1-10 sequentially

Option 10: Native ICS Protocol Scan (Step-by-Step)

This is the single most powerful standalone feature. When you select Option 10, the tool performs the following sequence:

1. **Target Expansion:** Expands CIDR ranges (e.g., 10.10.1.0/24 → 254 hosts) and IP ranges (e.g., 10.10.1.1-50 → 50 hosts) into individual host addresses.
2. **Port Scanning:** For each host, scans 60+ ICS-specific TCP and UDP ports using native Python sockets with configurable timeout and rate limits. Real-time progress bars display scan status.
3. **Protocol Detection:** For each open port, matches the port to the corresponding ICS protocol definition from the PROTOCOL_DEFS database.
4. **Protocol Enumeration:** Runs the protocol-specific native enumeration method (e.g., `modbus_read_device_id` for port 502, `s7_get_info` for port 102).
5. **Asset Creation:** Creates asset entries with discovered ports, services, vendor info, firmware versions, and protocol details.
6. **Finding Generation:** Generates findings with CVSS scores, ATT&CK TTP mappings, EMB3D properties, and compliance references.
7. **Results Export:** Saves all raw scan results to `native_scan_results.json` in the output directory.

INFO: Option 10 requires NO external tools. It works on air-gapped networks, fresh Windows installs, and minimal Linux containers.

Option 3: Shodan Lookup (Three-Tier Model)

Tier 1 (always runs, free, no key): Queries Shodan InternetDB API for each target IP. Returns open ports, CVEs, hostnames, and tags. No rate limit.

Tier 2 (always runs): Displays and exports all 228 built-in ICS Shodan/Google dorks to a file for manual or automated follow-up.

Tier 3 (only if API key configured): Runs full Shodan host queries with enriched data including product names, versions, and full vulnerability lists.

Option 4: OSINT Dork Search

The OSINT dork search checks for internet connectivity. If internet access is available, the tool executes live searches using DuckDuckGo (no API key required), substituting engagement details (customer name, IP ranges, vendor names) from Phase 0 into dork templates. Results are saved to osint_results.json with hit counts per query. If no internet is available, dork queries are exported to a text file for manual follow-up.

Option 8: Import Scan Results

Supports three import formats: (1) Nmap XML — parses -oX output, extracts IPs, ports, services, MAC addresses, vendor names, creates asset entries; (2) Nessus .nessus — parses vulnerability scan results, extracts IPs, ports, plugin names, CVEs, severity levels; (3) PCAP — analyzes packet captures using tshark for ICS protocol traffic.

TIP: Imported data merges with (does not replace) data from native scans. Run native scans first, then import external results to supplement.

6. Phase 2: Enumeration

Deep protocol-specific enumeration of discovered services. This phase requires assets discovered in Phase 1.

■ **If you enter Phase 2 without any discovered assets, you will see: 'No assets discovered yet. Run Phase 1 (Reconnaissance) first.'**

Op t	Function	How It Works
1	Protocol-Specific Enumeration	Runs native Python scanners FIRST for all detected protocols, then Nmap NSE scripts if installed. Results merged.
2	Modbus Unit ID Brute Force	Tests Unit IDs 1-255 to find hidden Modbus slave devices. Uses native Modbus frames (FC 0x11). No Nmap needed.
3	BACnet Object Enumeration	Native BACnet Who-Is broadcast + ReadProperty for discovered devices (UDP 47808).
4	OPC UA Anonymous Check	Native OPC UA Hello → ACK handshake to detect unauthenticated endpoint access (TCP 4840).
5	SNMP Community Brute Force	Tests community strings: public, private, secret, SCADA, ics, control, monitor using native ASN.1/BER SNMP GET.
6	S7comm Block Upload	Native COTP → S7 Setup → SZL Read chain to extract Siemens PLC identity, serial number, firmware version.
7	MQTT Topic Discovery	Native MQTT CONNECT → CONNACK to detect anonymous broker access. Checks return code for auth requirements.
8	HTTP/HTTPS Interface Scan	Native urllib probe with 30+ ICS keyword detection (scada, hmi, plc, webvisu, factorytalk, etc.).

TIP: Option 1 runs ALL protocol scanners sequentially, so you typically don't need to run options 2-8 individually unless you want to focus on a specific protocol or re-run a specific check.

7. Phase 3: Vulnerability Analysis

Op t	Function	Prerequisites
1	Default Credential Check	Needs assets from Phase 1. Tests 228 embedded credentials matched by vendor + protocol + ICS port.
2	CVE Lookup (NVD)	Needs assets with vendor names. Queries NVD API. Without key: 1 query/6.5 sec. With key: 1 query/0.7 sec.
3	CWE Mapping	Needs assets. Maps CWE IDs based on detected protocols and port weaknesses.
4	EMB3D Property Mapping	Needs assets. Maps MITRE EMB3D threat properties (NET.PROTO, AUTH.REMOTE, etc.) based on open ports.
5	Nessus/OpenVAS Import	Works standalone — import .nessus XML from any Nessus/OpenVAS scan.
6	CVSS Risk Scoring	Needs findings. Calculates CVSS v3.1 base scores using 8 ICS-specific patterns (default_creds=9.8, no_auth=10.0, etc.).
7	STIG/CIS Benchmark Scan	Needs assets. Runs 66 native compliance checks. Exports SCAP/XCCDF XML. See Chapter 14 for full reference.

Default Credential Check (Step-by-Step)

1. The tool iterates through each discovered asset with a progress bar showing per-asset status. 2. For each asset, it matches the vendor name, protocol type, and open ports against the 228 embedded credential entries. 3. For each matching credential, it attempts authentication using the appropriate protocol handler (HTTP basic/form auth via urllib, Telnet login, SSH via paramiko/sshpas, FTP login, SNMP community string test, VNC auth). 4. Successful logins generate a Critical-severity finding (CVSS 9.8) with the credential pair, ATT&CK TTP T0812, and compliance references. 5. All results are logged to the engagement state for inclusion in reports.

STIG/CIS Benchmark Scan Overview

Phase 3 Option 7 runs 66 native configuration compliance checks organized into 12 categories. Each check maps to both a DISA STIG ID and a CIS Benchmark ID. Results export in SCAP/XCCDF XML format for integration with vulnerability management platforms like ACAS, Tenable.sc, or eMASS. See Chapter 14 for the complete 66-check reference table.

8. Phase 4: Exploitation

Phase 4 provides 'living off the land' exploitation using each protocol's own native write/control functions, plus integration with external exploit tools.

Op t	Function	Native?
1	Run Metasploit Module	No (needs msfconsole). Auto-lists applicable modules per asset.
2	Run Custom Payload	Yes (Python importlib). Executes user-provided .py/.rc/.nse scripts.
3	Protocol Function Code Attack	YES — 56 native exploit methods. See sub-menu below.
4	Default Credential Login	Yes (native urllib + socket). Tests known defaults against web interfaces.
5	Firmware Extraction (binwalk)	No (needs binwalk). Extracts firmware + entropy analysis + string search.
6	Hardware Debug (JTAG/UART/SWD)	Partial (JTAG needs OpenOCD, UART uses native pyserial).
7	Binary Analysis (GDB/radare2)	Partial (strings is native, binwalk/radare2 are optional).
8	Web Interface Exploit	Yes (native urllib with ICS keywords). nikto/nmap supplement.
9	Run Scapy Module	Yes (native importlib). Executes user Scapy scripts.
10	Proof of Compromise	Yes (native). Terminal screenshot + benign flag + HMI web capture + GUI screenshot.

Option 3: Protocol Function Code Attack Sub-Menu

When you select Option 3, a sub-menu appears with protocol-specific exploit operations. Discovered targets are displayed with their detected protocols for easy selection. Each operation uses actual protocol write/control commands via pure Python with struct.pack — not stubs.

Su b	Protocol Attack	Physical Impact
a	Modbus Write Coil (FC 0x05)	Opens/closes a valve, starts/stops a motor, energizes a relay
g	Modbus Write Register (FC 0x06)	Changes a setpoint, speed, temperature target, or timer
c	Modbus Unit ID Brute Force	Discovers undocumented PLCs, RTUs, or I/O modules on the bus
d	S7comm CPU Stop (0x29)	Halts Siemens PLC execution — process goes to safe/uncontrolled state
e	MQTT Publish to topic	Injects false sensor data, triggers actuators, sends rogue commands
f	HTTP Default Login Test	Gains authenticated access to HMI/config web interfaces
r	— Function Code Reference	Displays all function codes for all protocols with DANGEROUS flags

■ **These commands modify PHYSICAL PROCESS STATE. They can halt PLCs, change setpoints, trip breakers, open valves, and actuate outputs. Use ONLY with explicit written authorization and coordinate with plant operations.**

The full NativeScanner class has 56 exploit methods covering ALL 56 protocols. The sub-menu shows the most common ones. For other protocols, use the NativeScanner class directly in custom Python payloads (Phase 4 Option 2).

Proof of Compromise (Option 10)

Captures evidence for all discovered and exploited hosts automatically: (1) terminal screenshot of the current session state, (2) drops a benign flag file (ICS_PENTEST_FLAG.txt) on the target if writable access exists — full file path, filename, and timestamp are displayed and logged for post-engagement

cleanup, (3) downloads HTML pages and HTTP response headers from discovered web-based HMI interfaces on ports 80, 443, 8080, 8443 with ICS keyword detection, (4) GUI screenshot if available. All evidence is organized by target IP in the evidence/ subdirectory with timestamps and finding IDs.

9. Phase 5: Post-Exploitation

■ *Phase 5 requires assets discovered and (ideally) exploited in earlier phases. You will see a warning if no assets exist.*

Op t	Function	How It Works
1	Credential Harvesting	SSH login with discovered creds (tries paramiko first, sshpass fallback), HTTP config page scraping via native urllib. Extracts passwords, keys, config files.
2	Lateral Movement Mapping	Maps each compromised host to reachable hosts via shared open ports. Shows potential pivot paths based on network topology and protocol adjacency.
3	PLC Logic Download	Extracts PLC programs via protocol-specific Nmap scripts (e.g., s7comm-download-block) and MSF modules. Supports S7, AB, Modicon.
4	Configuration Extraction	Pulls device configurations across all detected protocols using native scanners and Nmap. Extracts running configs, firmware info, network settings.
5	Network Traffic Capture	Captures PCAP via tcpdump or tshark for offline protocol analysis in Wireshark. Filters for ICS-specific traffic on detected ports.

Lateral Movement Mapping Details

The lateral movement mapper analyzes the discovered network topology to identify potential pivot paths. For each compromised host, it checks which other hosts share open ports and protocols, and maps out paths an attacker could use to move laterally through the ICS network. This is particularly important in ICS environments where flat networks and shared protocols (like Modbus on port 502) can enable rapid lateral movement from IT to OT zones.

10. Phase 6: Threat Modeling

Attack and threat modeling based on MITRE ATT&CK for ICS TTPs and MITRE EMB3D properties and threats.

■ **Phase 6 requires findings generated in earlier phases. Without findings, threat models will be empty.**

Op t	Function
1	MITRE ATT&CK for ICS TTP Mapping — maps each finding to ATT&CK techniques (37 mapped)
2	MITRE EMB3D Threat Property Mapping — maps device properties (NET.PROTO, AUTH.REMOTE, etc.)
3	ICS Kill Chain Diagram — generates data-driven Graphviz DOT + Mermaid + interactive HTML diagrams
4	Threat Model Matrix — generates attack mapping matrix (XLSX + CSV) with full ATT&CK/EMB3D names
5	CALDERA OT Integration — pushes findings to CALDERA adversary emulation (needs CALDERA server)

Kill Chain Diagram Output

Option 3 generates an ICS Cyber Kill Chain diagram in three formats: Graphviz DOT (renderable to PNG/SVG if graphviz is installed), Mermaid (.mmd renderable in any Mermaid-compatible viewer or GitHub), and interactive HTML with embedded Mermaid.js CDN rendering. The diagram is fully data-driven from actual assessment findings — each stage shows the specific ATT&CK TTP IDs and names, affected assets, OSI layer annotations (L2/L3/L7), and direction paths between stages. The 7 stages are: Reconnaissance → Weaponization → Delivery → Exploitation → Installation → Command & Control → Actions on Objectives. If Graphviz is not installed, the text-based formats are generated without error.

Threat Model Matrix Output

Option 4 generates the attack mapping matrix in both XLSX (with openpyxl) and CSV formats. The XLSX workbook contains three tabs: (1) Attack Matrix — TTP ID, TTP Name (full), Tactic, EMB3D Properties, Finding ID, Title, Affected Asset, Protocol, Severity, CVSS Score, Evidence, Remediation; (2) TTP Reference — all 37 MITIGATION_MAP entries with full technique names and all 8 framework control references; (3) Executive Summary — project info, finding counts, severity distribution.

11. Phase 7: Mitigation Mapping

Maps each finding to controls from 8 compliance frameworks simultaneously. Framework references (Options 1-7) display even without findings — they show all 37 mapped TTPs and their control references.

Op t	Framework
1	ISA/IEC 62443 (Parts 2-1, 3-2, 3-3, 4-1, 4-2)
2	NIST SP 800-82 Rev 3 / SP 800-53 Rev 5
3	NERC CIP
4	MITRE D3FEND
5	ISA 84.00.09 v3 (Safety Instrumented Systems)
6	EU CRA / NIS2
7	AWWA (Water Sector)
8	Generate Compliance Crosswalk — all 8 frameworks in one table (needs findings)
9	Generate POA&M Roadmap — prioritized remediation plan with all 8 frameworks + XLSX output

Compliance Crosswalk (Option 8)

Generates a comprehensive table mapping each finding to controls across all 8 frameworks simultaneously. Each row contains: Finding ID, Title, Severity, CVSS Score, ATT&CK TTP, and the corresponding control reference from each framework (ISA 62443, NIST 800-82, NIST 800-53, NERC CIP, D3FEND, ISA 84, EU CRA/NIS2, AWWA).

POA&M Roadmap (Option 9)

Generates a prioritized Plan of Action and Milestones (POA&M) with remediation timelines based on CVSS severity. Critical findings get 24-48 hour remediation windows, High gets 1-2 weeks, Medium gets 30-60 days, and Low gets 90 days. The POA&M includes all 8 compliance framework control references per finding with detailed section numbers (e.g., AWWA J100-13 Sec 6.1, NIS2 Art. 21(2)(a), EU CRA Art. 10(1), ISA 84.00.09 Sec 11). Both Markdown and XLSX formats are generated with severity color-coding.

12. Phase 8: Reporting

Op t	Format	Generator	Notes
1	Markdown (.md)	Native Python	Plain text, version-control friendly
2	HTML (.html)	Native Python + CSS	Professional styling, embedded SVG diagrams
3	DOCX (.docx)	Native Python (zipfile + OOXML)	Opens in Word, LibreOffice, Google Docs
4	PDF (.pdf)	Native Python (PDF 1.4 text)	Opens in any PDF viewer
5	All Formats	All above simultaneously	Each format wrapped independently
6	Attack Matrix XLSX	Native + openpyxl	ATT&CK for ICS + EMB3D with full names
7	MSF Resource Script (.rc)	Native	Auto-generated for applicable MSF modules
8	Save Session (JSON)	Native	Save state for resume. Also auto-saves after every action.

All report formats are generated natively using only Python stdlib. No pandoc, wkhtmltopdf, or LaTeX installation needed.

■ *If no findings exist, the tool will ask 'Generate empty report anyway?' before proceeding. This prevents accidental blank report generation.*

Report Contents

Each full report includes: Executive Summary, Scope & Methodology, Asset Inventory, Findings Detail (with CVSS scores, ATT&CK TTPs, evidence, remediation), ICS Kill Chain Threat Model, Compliance Crosswalk, POA&M Roadmap, and Appendices (protocol reference, credential database summary, tool configuration).

13. Native Protocol Scanner Reference

The NativeScanner class provides 121 pure Python methods: 60 enumeration (read-only, safe for ICS), 56 exploitation (write/control — require authorization), and 5 utility methods. These implement actual protocol handshakes using socket + struct.

Enumeration Methods (60) — Read-Only, Safe for ICS

Protocol	Method	What It Reads
Modbus/TCP	modbus_read_device_id	Vendor, Product, Revision via FC 0x2B MEI
Modbus/TCP	modbus_report_slave_id	Slave ID, Run/Stop status via FC 0x11
Modbus/TCP	modbus_read_registers	Holding register values via FC 0x03
Modbus/TCP	modbus_read_coils	Coil ON/OFF states via FC 0x01
Modbus/TCP	modbus_scan_unit_ids	Active slaves via 1-255 brute force
S7comm	s7_get_info	Module type, serial, firmware via COTP → S7 → SZL chain
EtherNet/IP	enip_list_identity	Vendor ID, product name, serial number
BACnet	bacnet_whois	Device ID, vendor, model via Who-Is broadcast
BACnet	bacnet_read_property	Object properties (name, vendor, etc.)
DNP3	dnp3_read_info	Protocol confirmation, transport layer probe
PROFINET	profinet_dcp_identify	Station name, IP address, vendor via DCP Identify All
OPC UA	opcua_get_endpoints	Protocol version, endpoint info via Hello → ACK
OPC DA	opcda_probe	DCOM/RPC endpoint mapper detection on port 135
IEC 61850 MMS	iec61850_mms_probe	COTP → MMS Initiate, device identity strings
IEC 61850 GOOSE	iec61850_goose_capture	Layer 2 frame capture via raw socket (tshark fallback)
IEC 61850 SV	iec61850_sv_capture	Layer 2 Sampled Values via raw socket (tshark fallback)
IEC 104	iec104_startdt	STARTDT act → confirm handshake
IEC 101	iec101_probe	Serial interrogation via pyserial
FINS (Omron)	fins_cpu_read	CPU model, firmware via command 0x0501
MELSEC	melsec_read	PLC device memory via MC Protocol 3E binary
GE SRTP	ge_srtp_read_info	Init + device info extraction
PCCC	pccc_read_info	Legacy AB device info via ENIP encapsulation
TriStation	tristation_get_status	SIS controller state via Get CP Status (FC 0x07)
SNMP	snmp_get	Native ASN.1/BER encoded GET (no pysnmp needed)
MQTT	mqtt_connect_probe	Anonymous access detection via CONNECT → CONNACK
CoAP	coap_discover	Resource list via GET /.well-known/core (RFC 7252)
HART-IP	hart_ip_read_uid	Device type, revision via Command 0 + HART checksum
ROC/ROC Plus	roc_read_info	RTU config via Opcode 6 + CRC-16 checksum
Fox/Niagara	fox_hello	Station name, version, all key-value data
VNC	vnc_probe	RFB version, auth types, no-auth detection
RDP	rdp_probe	X.224 handshake, NLA requirement detection
SSH	ssh_probe	Banner, protocol version, software identification

Plus 28 more methods covering: CC-Link, EtherCAT, POWERLINK, DLMS, AMQP, Yokogawa, Crimson, ProConOS, KNX, LonWorks, IO-Link, CANopen, PROFIBUS, WirelessHART, Telnet, FTP, TFTP, HTTP/S, Moxa, MQTT-SN, LoRaWAN, ZigBee, Z-Wave, BLE, ICCP/TASE.2, FF-HSE, Modbus RTU.

Utility Methods (5)

Method	Description
<code>native_arp_scan()</code>	Layer 2 ARP-based host discovery. Uses arp-scan (Linux) or arp -a fallback (cross-platform). Discovers MAC addresses and vendor OUIs at the data link layer.
<code>native_ssl_enum()</code>	SSL/TLS cipher and certificate enumeration. Checks TLS version, cipher suite, certificate details (subject, issuer, expiry, serial number), weak cipher detection, and weak TLS version detection. Uses Python ssl module.
<code>native_dns_enum()</code>	DNS enumeration with A record lookups and ICS-specific subdomain brute forcing (scada, hmi, plc, opc, historian, dcs, rtu, modbus, bacnet, eng, ops, remote, api, dev, test, monitor, siem). Uses Python socket module.
<code>native_owasp_basic()</code>	Basic OWASP Top 10 web vulnerability checks: missing security headers (HSTS, CSP, X-Frame-Options, X-Content-Type-Options, X-XSS-Protection), server version disclosure, directory listing detection, reflected XSS test, and path traversal test. Uses urllib.
<code>enumerate_host()</code>	Master enumeration dispatcher. For each host, checks all open ports against PROTOCOL_DEFS and runs the appropriate protocol-specific enumeration method. Returns a list of all results.

Exploitation Methods (56) — Require Written Authorization

Every protocol with write/control/change functions has a native exploit method. These use actual protocol frames built with struct.pack — not stubs. Each method gracefully handles unreachable hosts without crashing. Key exploit methods include: `modbus_write_coil`, `modbus_write_register`, `s7_stop_cpu`, `dnp3_direct_operate`, `dnp3_cold_restart`, `bacnet_write_property`, `bacnet_reinitialize`, `profinet_dcp_set`, `enip_write_tag`, `opcua_write`, `iec61850_mms_write`, `iec61850_goose_spoof`, `iec61850_sv_inject`, `iec104_single_command`, `iec101_single_command`, `fins_memory_write`, `fins_run_stop`, `ge_srtip_write_memory`, `pccc_write_data`, `tristation_halt_program`, `melsec_write`, `mqtt_publish`, `mqtt_sn_publish`, `coap_put`, `amqp_publish`, `snmp_set`, `hart_ip_write`, `roc_write_config`, `fox_invoke`, `crimson_write`, `dlms_write_register`, `yokogawa_write`, `proconos_write`, `knx_write_group`, `lonworks_write`, `iccp_write`, `canopen_sdo_write`, `wirelesshart_write`, `ethercat_write`, `powerlink_write_sdo`, `ff_hse_write`, `moxa_config_write`, `telnet_login`, `ssh_login`, `ftp_upload`, `tftp_write`, `vnc_auth_bypass`, `rdp_login`, `opcda_write`, `ble_write_characteristic`, `zigbee_send_command`, `zwave_send_command`, `lorawan_send_downlink`, `iolink_write_param`, `http_default_login`.

14. STIG/CIS Benchmark Scanner Reference

Phase 3 Option 7 runs 66 native configuration compliance checks organized by platform and device type. Each check maps to both a DISA STIG ID and a CIS Benchmark ID. Results export in SCAP/XCCDF XML format.

Network Device Checks (13)

Check	STIG ID	CIS ID	Severity
Default/Factory Credentials Not Changed	SRG-APP-000516-NDM-000340	CIS Network L1 1.1.1	Critical
Telnet Service Enabled (Cleartext)	SRG-APP-000142	CIS Network L1 2.1.1	High
SNMPv1/v2c Enabled (No Encryption)	SRG-APP-000142	CIS Network L1 3.1.1	High
HTTP Without TLS	SRG-APP-000014	CIS Network L1 2.2.1	High
FTP Service Enabled (Cleartext)	SRG-APP-000142	CIS Network L1 2.1.3	Medium
ICS Protocol Without Authentication	SRG-APP-000516	CIS ICS 1.1	Critical
ICS Device Internet-Exposed	SRG-NET-000019	CIS ICS 2.1	Critical
Firmware/Software Not Current	SRG-APP-000456	CIS ICS 3.1	Medium
RDP Exposed Without NLA	V-220933	CIS Win10 L1 18.9.65	High
SMBv1 Protocol Enabled	V-220729	CIS Win10 L1 18.3.3	High
SSH Root Login Permitted	V-230330	CIS RHEL8 L1 5.2.10	Medium
Weak SSH Ciphers/MACs Enabled	V-230253	CIS RHEL8 L1 5.2.13	Medium
VNC Without Authentication	SRG-APP-000516	CIS Remote Access 1.1	Critical

Additional check categories: Windows Server/Workstation (12 checks), Linux/RHEL/Ubuntu (9 checks), RTOS/Firmware (4 checks), Vendor-Specific including Cisco, FortiGate, Palo Alto, Juniper (11 checks), and Additional checks covering SQL Server, Web Servers, VMware ESXi, Active Directory, DNS, Siemens SCALANCE, IoT devices, and Windows UAC (17 checks). All benchmark results automatically export to SCAP/XCCDF XML format importable into ACAS, Tenable.sc, DISA eMASS, or SCAP Compliance Checker (SCC).

15. Covert Communications & Exfiltration

For authorized red team campaigns. Provides 8 tunnel types and 11 exfiltration destinations.

Supported Tunnel Types (8)

Channel	Description	External Tool
TOR	Route traffic through TOR SOCKS proxy (port 9050)	tor binary
DNS Tunneling	Tunnel data via DNS TXT/CNAME queries	iodine / dnscat2
ICMP Tunneling	Tunnel data inside ICMP echo request/reply	ptunnel / icmpsh
HTTP/S Tunneling	Reverse proxy tunnel	chisel / ngrok
SSH SOCKS Proxy	SSH dynamic port forwarding (-D 1080)	ssh (built-in)
Tailscale VPN Mesh	Zero-config WireGuard mesh network	tailscale
WireGuard VPN	Fast kernel-level VPN tunnel	wg / wg-quick
Steganography	Hide data inside image files using LSB encoding	None (pure Python)

Exfiltration Destinations (11)

HTTP/S POST, FTP, DNS TXT Records, SMB Share, OneDrive, Google Drive, AWS S3, Dropbox, SMTP Email, ICMP Payload, Local Encrypted Archive.

TIP: Steganography is the only tunnel that requires no external tools. It uses pure Python LSB (Least Significant Bit) embedding to hide data inside PNG/BMP image files.

16. Network Infrastructure Database

39 vendor entries covering firewalls, routers, switches, gateways, wireless access points, serial servers, VPN gateways, DCS/PLC systems. Each entry includes default credentials, admin web paths, known CVEs, and ICS relevance notes.

Firewalls (7): Palo Alto, Fortinet FortiGate, Cisco ASA/Firepower, Check Point, Juniper SRX/EX, Siemens SCALANCE S/M, Hirschmann EAGLE

Routers (4): Cisco IE (IR1101/IR8340), Sierra Wireless, Moxa EDR, Pepwave/Peplink

Switches (12): Cisco IE, Siemens SCALANCE X/XC/XR, Rockwell Stratix, Red Lion N-Tron, Hirschmann MACH/RSP/MICE, Phoenix Contact FL, Netgear ProSafe/Smart, Ubiquiti EdgeSwitch/UniFi, TP-Link, Aruba/HPE, Belden (GarrettCom), Westermo Lynx/Wolverine

Gateways (5): Moxa MGate, HMS Anybus, Red Lion Sixnet, Digi ConnectPort, B&B Electronics (Advantech)

DCS/PLC (2): Emerson DeltaV, AutomationDirect ProductivityOpen/BRX

Wireless/RF (5): Cisco IW, Siemens SCALANCE W, Rajant Kinetic Mesh, FreeWave/CalAmp, MDS Orbit (GE)

Serial Servers (2): Moxa NPort, Lantronix UDS/xDirect/SLC

VPN (2): Fortinet FortiGate SSL-VPN, Pulse Secure/Ivanti

Each vendor entry in the NETWORK_INFRA database contains: device type, default TCP/UDP ports, default username/password pairs, admin web interface paths, known CVE IDs with severity, supported ICS protocols, and ICS relevance notes describing typical deployment in industrial environments.

17. External Tool Integrations

All external tool integrations are optional. The tool auto-detects availability at startup (shown via T command). Native features ALWAYS execute first.

Tool	Priority	Notes
Nmap	Native scanner first, Nmap augments	NSE scripts add depth to native protocol probes
Metasploit	Native exploits first, MSF supplements	MSF modules auto-listed from PROTOCOL_DEFS database
Nessus/ OpenVAS	Import only (Phase 1 Opt 8)	Merges with native scan data, does not replace
Shodan	InternetDB (free) first, API supplements	InternetDB needs no key; full API adds product/version
Censys	Optional supplement	Internet-wide scan data for target enrichment
NVD API	Always native urllib	Without key: 6.5s rate limit. With key: 0.7s
CALDERA	Optional push	Needs server URL + API key in Phase 0
Claude/ OpenAI	Optional AI analysis	AI-assisted finding classification and reporting
tshark	Native raw socket first for GOOSE/SV	Fallback when raw socket requires root
paramiko	Tried first for SSH login	Falls back to sshpass if not installed

18. Data Sources & Customization

Embedded Data (built-in, no external files)

- 228 default credential sets from ScadaStrangeLove + ICS OSINT research
- 129 Shodan ICS search dorks
- 99 Google/SCADA dorks
- 62 pentest checklist tasks across 9 categories
- 47 attack test cases from threat model template

Adding Custom Data

```
python3 ICSOTpentestv2.py --data-file my_custom_creds.json
```

Custom data supplements (does not replace) the embedded data. JSON format: {"default_credentials": [{"v": "Vendor", "d": "Device", "c": "admin:pass", "p": "80/tcp", "t": "Type", "pr": "Protocol"}]}

XLSX credential files can be loaded interactively via Phase 0 > Option 8 (requires openpyxl).

Modifying Native Features (Advanced Users)

- Add protocol handlers to NativeScanner — follow existing method pattern with _tcp_send_recv/_udp_send_recv
- Add MITIGATION_MAP entries — dict with keys: name, isa62443, nist, nerc_cip, defend, isa84, awwa, eu_cra, nis2
- Add NETWORK_INFRA_DEVICES entries — dict with type, ports, default_creds, admin_paths, known_cves, protocols, ics_relevance
- Add BenchmarkScanner.CHECKS entries — dict with title, stig, cis, severity, cvss, description, remediation, check_type
- Update embedded data — modify _EMBEDDED_DATA_B64 with new base64-encoded zlib-compressed JSON

19. Command-Line Interface Reference

Flag	Description
--target IP/CIDR	Target IP, CIDR range, or hostname
--protocols LIST	Comma-separated protocol names (e.g., modbus_tcp,s7comm)
--data-file PATH	Load additional JSON to supplement embedded data
--output-dir DIR	Output directory (default: auto-named from project name or target IP)
--list-protocols	List all 56 supported protocols and exit
--list-creds	Display embedded default credential database
--auto	Run all phases automatically (non-interactive)
--no-color	Disable ANSI color output
--os-detection	Enable OS detection in nmap scans
--aggression 1-5	Scan aggression level (1=paranoid, 5=insane)
--full-port-scan	Scan all 65535 TCP ports (default: ICS ports + top 1000)
--top-ports N	Scan top N ports in addition to ICS-specific ports
--debug	Enable verbose debug logging to activity log file
--log-file PATH	Specify path for activity log file (default: icsot_activity.log)
--resume PATH	Resume from a saved session JSON file
--session-dir PATH	Directory for session auto-save files

Output Directory Naming

The output directory is automatically named based on the engagement context. If a project name is set in Phase 0, the directory is named `ics_pentest_[project_name]`. If no project name is set but target IPs are configured, the directory uses `ics_pentest_[first_target_IP]`. If neither is available, it falls back to `ics_pentest_[timestamp]`. This ensures that multiple concurrent engagements produce uniquely identifiable output directories. The directory can always be overridden with the `--output-dir` flag.

20. AI-Assisted Analysis

ICSOTpentest.py supports optional integration with AI language models (Claude by Anthropic or OpenAI) for AI-assisted analysis of findings and reports. This feature is entirely optional and requires an API key.

Configuration

Configure via Phase 0 > Option 6 or using the set command:

```
set ai_key YOUR_API_KEY # Set your API key # Provider is auto-detected (anthropic or openai)
```

What AI-Assisted Analysis Provides

- Enhanced finding descriptions with ICS-specific context
- Risk prioritization recommendations based on process impact
- Remediation guidance tailored to the specific ICS vendor and protocol
- Executive summary generation with business-impact language
- Automated correlation of findings with known ICS attack campaigns

INFO: AI analysis is processed locally through API calls. No assessment data is stored by the AI provider beyond the API call duration. All findings and reports remain under your control.

21. Troubleshooting

Issue	Solution
License verification fails	Ensure cryptography is installed (pip install cryptography). Check .lic file is in same folder as script or ICSOT_LICENSE_FILE env var is set correctly.
No assets discovered	Verify target IPs are reachable (ping test). Use Phase 1 Option 10 (native scan) which requires no external tools. Check firewall rules.
Phase shows 'No assets' warning	Run Phase 1 first, or add targets in Phase 0 > Option 1, or import scan data via Phase 1 > Option 8.
Sub-menu options not visible	Some options only show details after prerequisite data exists. Follow the phase order (0 → 1 → 2 → 3...) for best results.
Can't back out of a sub-menu	Press 'b' for back. If stuck in a running operation, press Ctrl+C to interrupt gracefully, then 'b'. Tool won't crash.
Ctrl+C doesn't stop scanning	Cancellation takes effect at the next checkpoint. Wait a moment for the current network operation to timeout, then the tool returns to menu.
Permission denied on scan	Some ports require root/admin. Use sudo on Linux, Run as Administrator on Windows. ICS ports (502, 102) typically don't need elevated privileges. Layer 2 scans need root.
SNMP not responding	Verify UDP 161 is not blocked by firewall. Try different community strings via Phase 2 Option 5.
Modbus timeout	Check Unit ID — scan 1-255 with Phase 2 Option 2. Increase timeout in Phase 0 > Option 9. Verify port 502 is open.
Reports not generating	Check output directory is writable. All reports are native — no external tools needed. Use Phase 0 > Option 10 to set output directory.
DOCX won't open in Word	Verify file isn't corrupted. Native DOCX generator uses OOXML ZIP format — works in Word 2007+, LibreOffice, Google Docs.
PDF text looks plain	Native PDF uses Courier font for maximum compatibility. For styled PDF, use the HTML report + browser print to PDF.
Colors not displaying	Use --no-color flag. Or use a terminal that supports ANSI: Windows Terminal, PowerShell 7, iTerm2, any modern Linux terminal.
NVD API slow	Without API key: 1 query per 6.5 sec (NVD rate limit). Get a free key at https://nvd.nist.gov/developers/request-an-api-key
GOOSE/SV capture fails	Layer 2 protocols need root + raw sockets (Linux only) or tshark. Not available on Windows without Npcap.
External tool not detected	Tool checks PATH at startup. Verify tool is installed and accessible via command line. Use T command to see status.
Benchmark scan finds nothing	Benchmark checks require assets with specific characteristics (OS type, vendor, open ports). Run Phase 1 first.
AI analysis returns errors	Verify API key is correct. Check network connectivity to API endpoint. Ensure the AI provider service is available.
Session restore fails	Ensure the JSON session file was not manually edited. Must be valid JSON. Check file permissions.
Custom payload won't load	Payload must be a Python file with an execute(target, port, config) or craft_packet() function. Check for syntax errors.
Output folders all the same name	Set the project name in Phase 0 > Option 5. The output folder is automatically named using the project name or first target IP.

22. Appendix: Complete Protocol & Port Reference

56 protocols supported with native enumeration and exploitation methods.

#	Protocol	Port	Transport	Enum	Exploit
1	Modbus/TCP	502	TCP	5	2
2	DNP3	20000	TCP	1	2
3	EtherNet/IP CIP	44818	TCP	2	1
4	S7comm (Siemens)	102	TCP	1	1
5	BACnet	47808	UDP	2	2
6	PROFINET DCP	34964	UDP	1	1
7	OPC UA	4840	TCP	1	1
8	OPC DA (DCOM)	135	TCP	1	1
9	IEC 61850 MMS	102	TCP	1	1
10	IEC 61850 GOOSE	L2 0x88B8	Eth	1	1
11	IEC 61850 SV	L2 0x88BA	Eth	1	1
12	IEC 60870-5-104	2404	TCP	1	1
13	IEC 60870-5-101	Serial	RS-232/485	1	1
14	FINS (Omron)	9600	UDP	1	2
15	MELSEC (Mitsubishi)	5007	TCP	1	1
16	GE SRTP	18245	TCP	1	1
17	PCCC (Allen-Bradley)	44818	TCP	1	1
18	TriStation (Triconex)	1502	TCP	1	1
19	CC-Link IE	61414	UDP	1	1
20	Fox/Niagara	1911	TCP	1	1
21	Crimson v3 (Red Lion)	789	TCP	1	1
22	MQTT	1883	TCP	1	1
23	MQTT-SN	1883	UDP	1	1
24	CoAP	5683	UDP	1	1
25	LwM2M	5683	UDP	(CoAP)	(CoAP)
26	AMQP	5672	TCP	1	1
27	SNMP v1/v2c	161	UDP	1	1
28	HART-IP	5094	TCP	1	1
29	WirelessHART	5094	TCP (gw)	1	1
30	ROC/ROC Plus	4000	TCP	1	1
31	DLMS/COSEM	4059	TCP	1	1
32	EtherCAT	34980	UDP/L2	1	1
33	POWERLINK	3819	UDP/L2	1	1
34	FF-HSE	1089	UDP	1	1
35	Yokogawa Vnet/IP	48898	TCP	1	1
36	ProConOS	20547	TCP	1	1
37	Moxa NPort	4800	UDP	1	1
38	ICCP/TASE.2	102	TCP (MMS)	1	1
39	KNX (EIBnet/IP)	3671	UDP	1	1
40	LonWorks	1628	UDP	1	1

41	IO-Link	80/6789	HTTP/REST	2	1
42	CANopen	CAN bus	SocketCAN	1	1
43	PROFIBUS	Serial	RS-485	1	—
44	ZigBee	Serial	USB dongle	1	1
45	Z-Wave	Serial	USB ctrl	1	1
46	BLE	—	Bluetooth	1	1
47	LoRaWAN	1700	UDP/Serial	1	1
48	Telnet	23	TCP	1	1
49	SSH	22	TCP	1	1
50	FTP	21	TCP	1	1
51	TFTP	69	UDP	1	1
52	HTTP	80	TCP	1	1
53	HTTPS	443	TCP	1	1
54	RDP	3389	TCP	1	1
55	VNC	5900	TCP	1	1
56	Modbus RTU	Serial	RS-485	(TCP)	(TCP)

Copyright © April 2026 Isiah Jones. All Rights Reserved. v2.1 | <https://isiahjonesstem.github.io/security-methodology/> | isiahjones@mail.com